

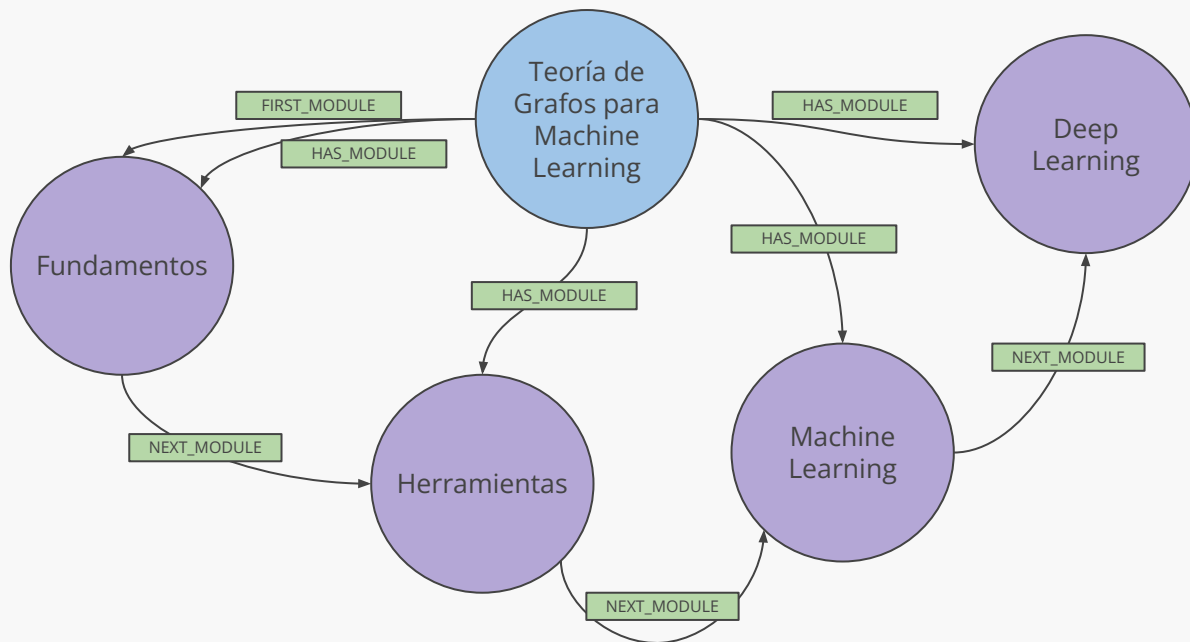
Clase 5.1

Machine learning para grafos

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl

Contenidos del curso

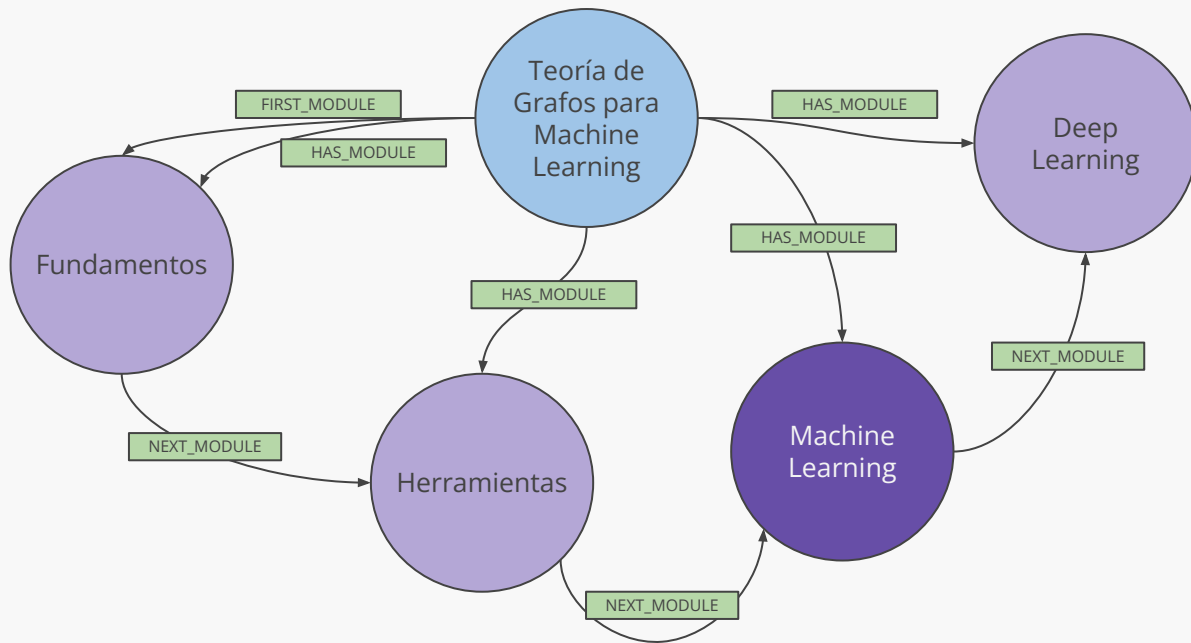


Contenidos del curso

3. Machine Learning

A partir de analítica de grafos, construir features que podamos usar en modelos de ML...

... con el objetivo de entender cómo se aplican conceptos de ML sobre grafos

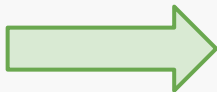


El problema es su estructura “dinámica”

Audio



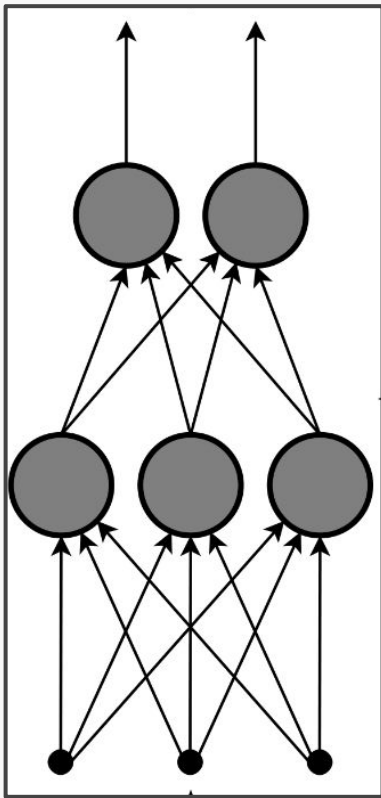
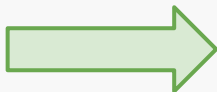
WaveNet, Wav2Vec, ...



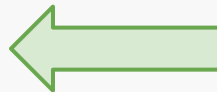
Texto



Word embeddings,
Seq2Seq, Transformers, ...



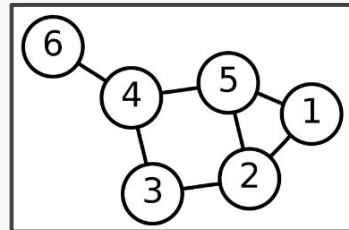
CNNs, Vision Transformers, ...



Imágenes



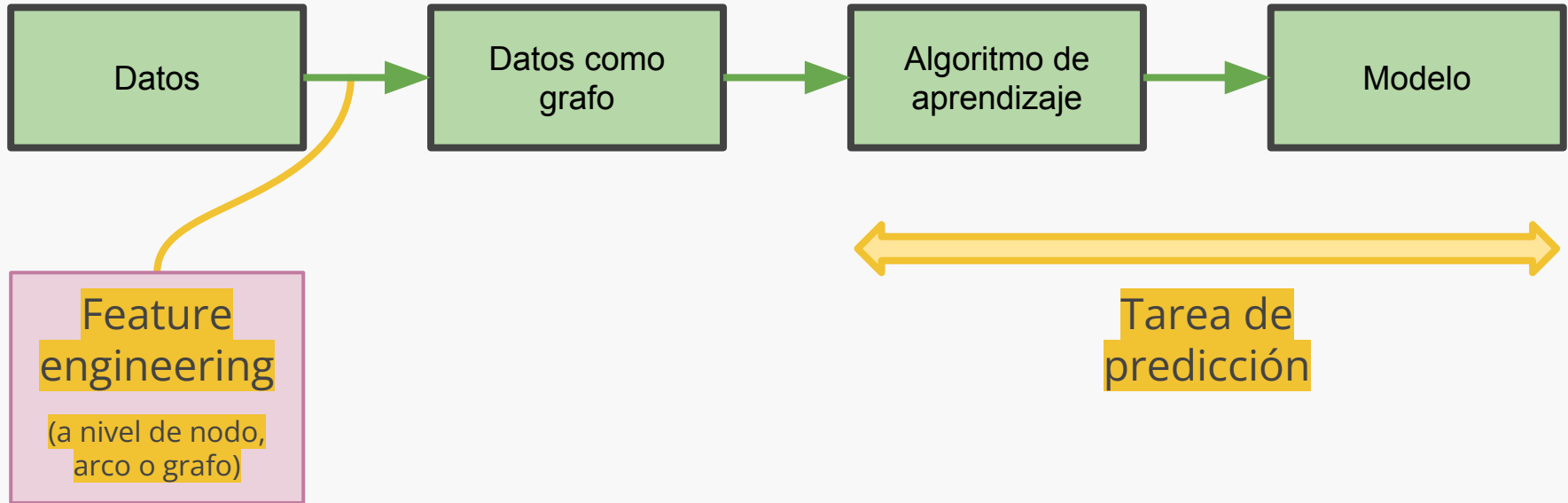
Grafos



???

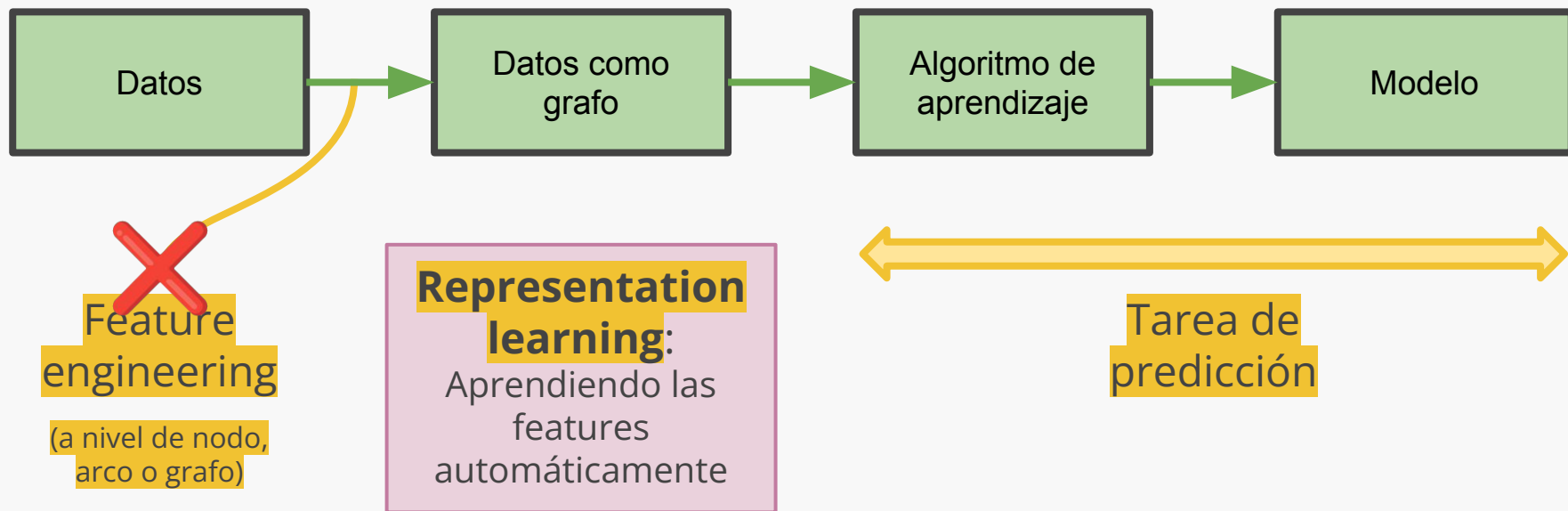
Como en todo problema de Machine Learning...

Dadas las *features* de un grafo (o porción), aprender un modelo que convierte las *features* en la salida esperada:



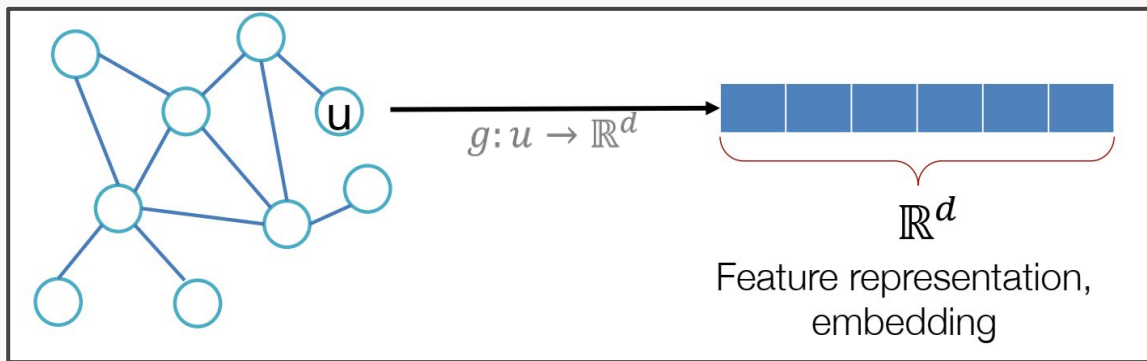
Aprender la representación de un grafo

Para evitar la necesidad de hacer *feature engineering* para cada tarea que queramos realizar, podemos aprender la representación de un grafo:



Aprender la representación de un grafo

Objetivo: Aprender una representación independiente de la tarea en cuestión para aplicar ML sobre grafos

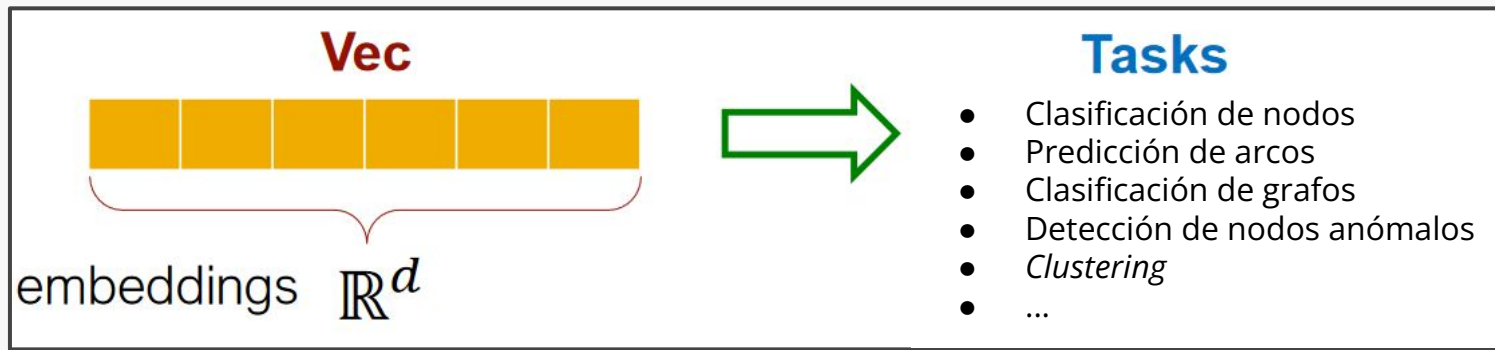


Podemos ver también esto como el aprendizaje de un mapeo de nodos a un espacio de embedding, donde nodos similares sean cercanos en esta espacio

¿Por qué usar *embeddings*?

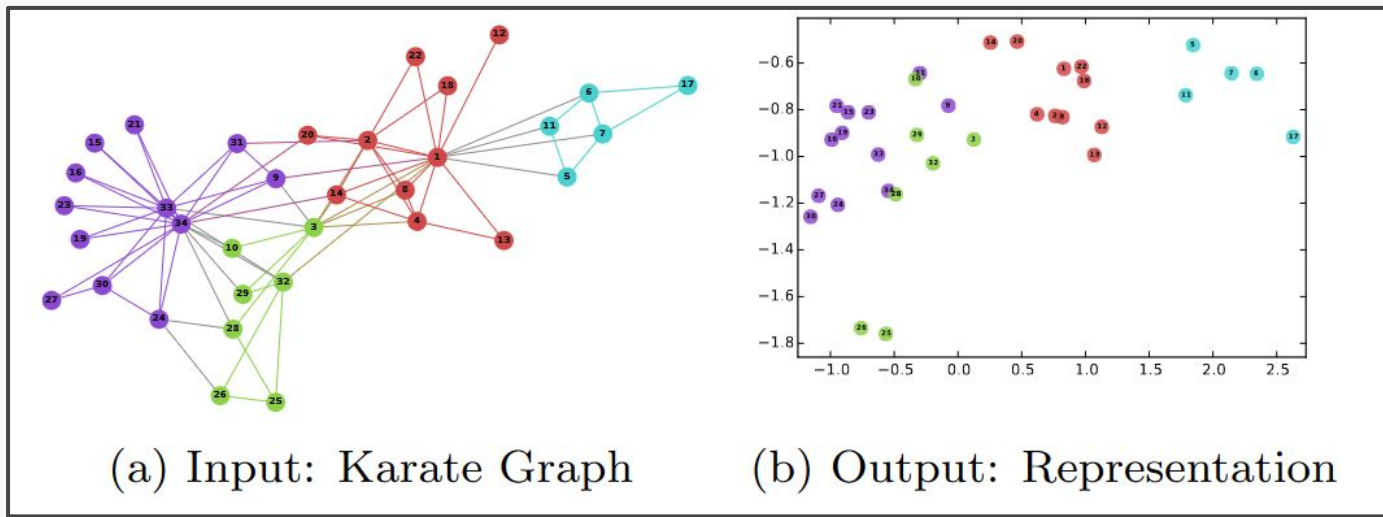
Tarea: Llevar nodos a un espacio latente (representados por un embedding):

- Similaridad de *embeddings* indica similaridad en grafo (ej: existe un arco)
- Codifican información del grafo
- Potencialmente usables en múltiples tareas



Ejemplo: Representar interacciones sociales

Podemos interesarnos en un espacio de embedding, donde nodos “similares” (según la estructura del grafo) se encuentren cerca en el espacio.



Agenda

✨ *Embeddings* de nodos

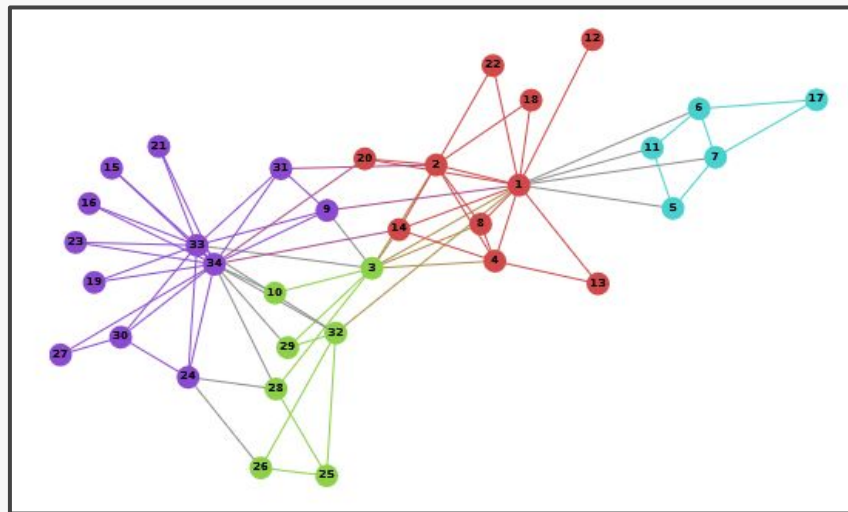
Embeddings de grafos

Comencemos formalizando conceptos

Contamos con:

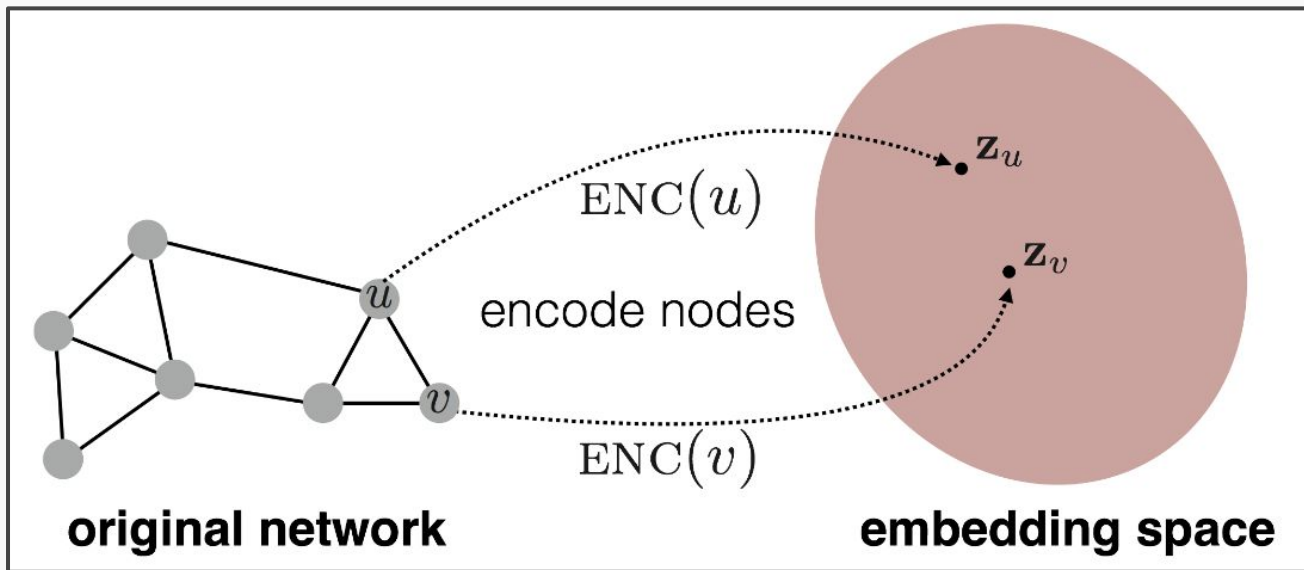
- Un grafo G
- V es el conjunto de nodos
- A es la matriz de adyacencia

No pensemos en *features* ni en usar información adicional

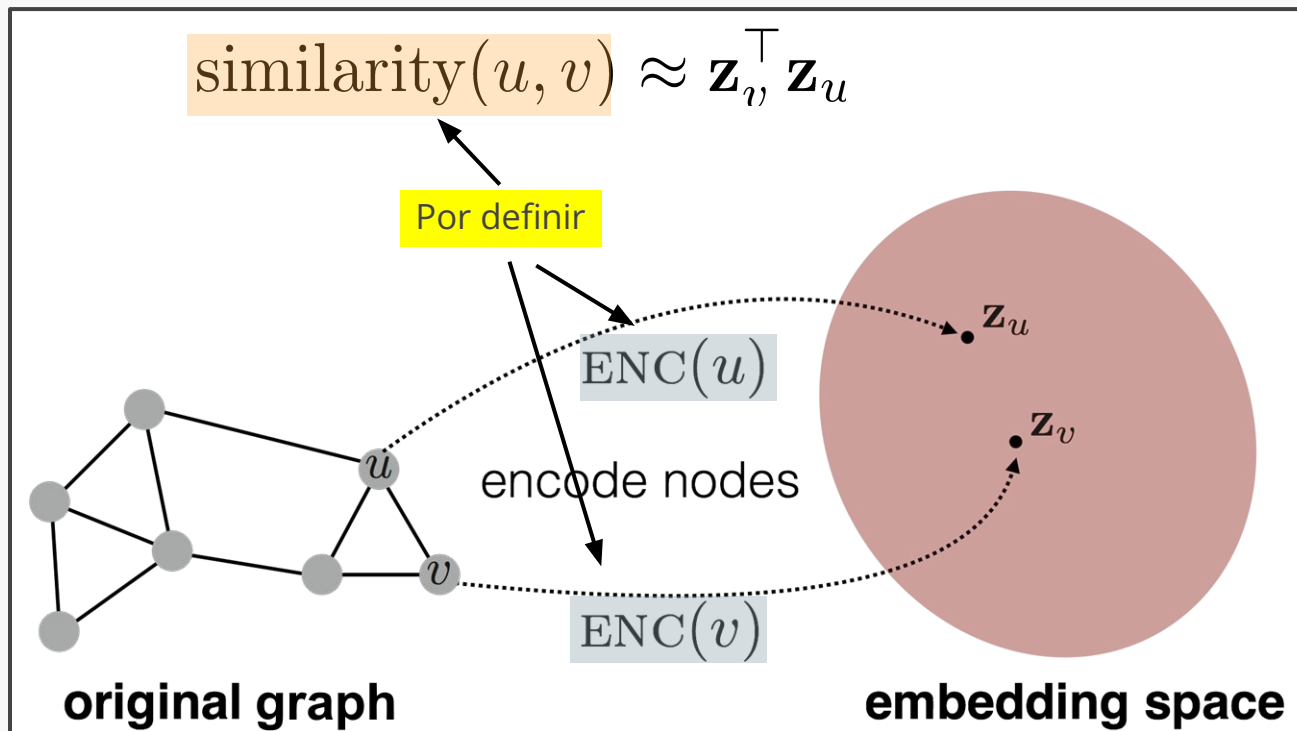


Embedding de nodos (*node embeddings*)

El objetivo es codificar los nodos del grafo de forma que la **similaridad en el espacio de *embedding*** (medida como el producto punto) aproxima la **similaridad en el grafo original**



Embedding de nodos (*node embeddings*)



Aprendiendo un *node embedding*

1. Encoder **ENC** mapea de nodos a *embeddings*
2. Definimos una función de similitud **similarity**, es decir, una medida de similitud entre nodos en el grafo original
3. Decoder **DEC** mapea de *embeddings* a un *score* de similitud
4. Optimizamos los parámetros del encoder de manera que:

$$\text{similarity}(u, v) \approx \mathbf{z}_v^T \mathbf{z}_u$$

Similaridad en la red original

Similaridad del *embedding*

Necesitamos estos dos elementos

- **Encoder ENC**: mapea cada nodo a un vector de baja dimensionalidad

$$\text{ENC}(v) = \mathbf{z}_v$$

Diagram illustrating the Encoder ENC function. The equation $\text{ENC}(v) = \mathbf{z}_v$ is shown. An arrow points from the variable v to the text "Nodo del grafo". Another arrow points from the vector $\mathbf{z}_v$ to the text "Embedding d-dimensional".

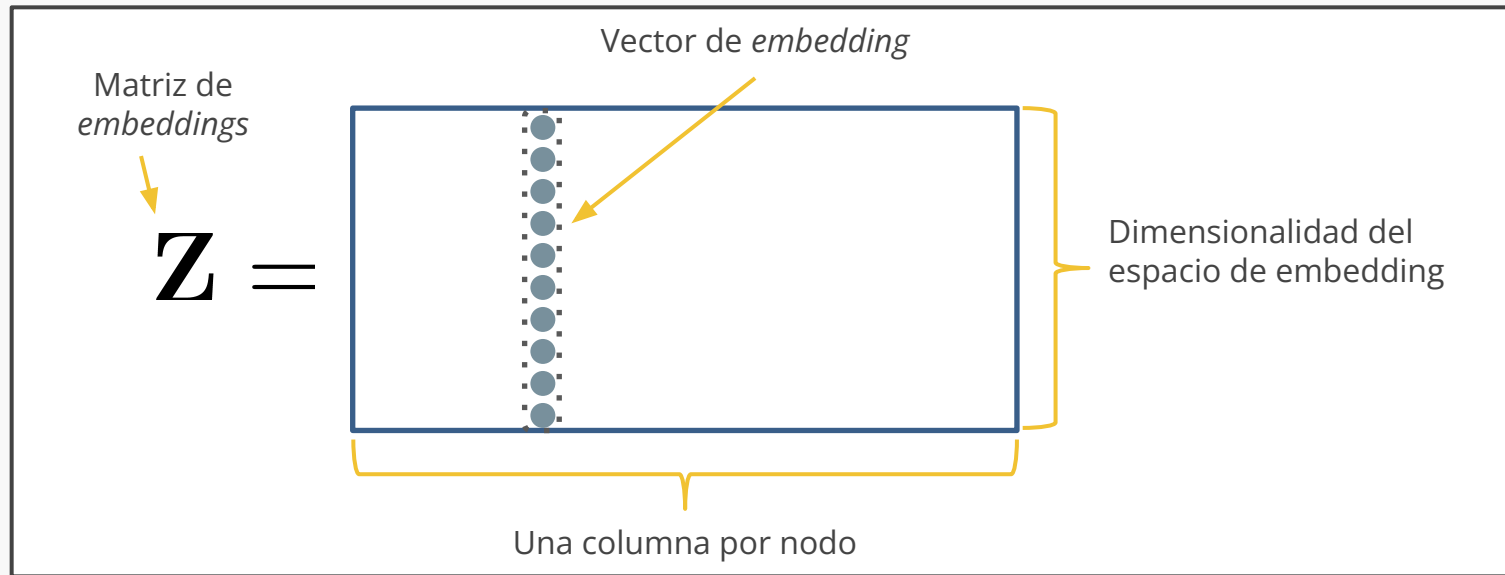
- **Función de similitud similarity**: especifica cómo se mapean las relaciones del grafo original al espacio de embedding

$$\text{similarity}(u, v) \approx \mathbf{z}_v^\top \mathbf{z}_u$$

Diagram illustrating the similarity function. The equation $\text{similarity}(u, v) \approx \mathbf{z}_v^\top \mathbf{z}_u$ is shown. An arrow points from the function $\text{similarity}(u, v)$ to the text "Similitud entre u y v en el grafo". Two arrows point from the vectors $\mathbf{z}_v$ and $\mathbf{z}_u$ to the text "Producto punto entre *embeddings* de nodos".

Encoding simple

El **encoder** más simple es simplemente indexar un vector en una matriz de *embeddings*, es decir, tomar las *features* desde un *embedding* ya definido:



Encoding simple

El **encoder** más simple es simplemente indexar un vector en una matriz de *embeddings*, es decir, tomar las *features* desde un *embedding* ya definido:

$$\text{ENC}(v) = \mathbf{z}_v = \mathbf{Z} \cdot \mathbf{v}$$

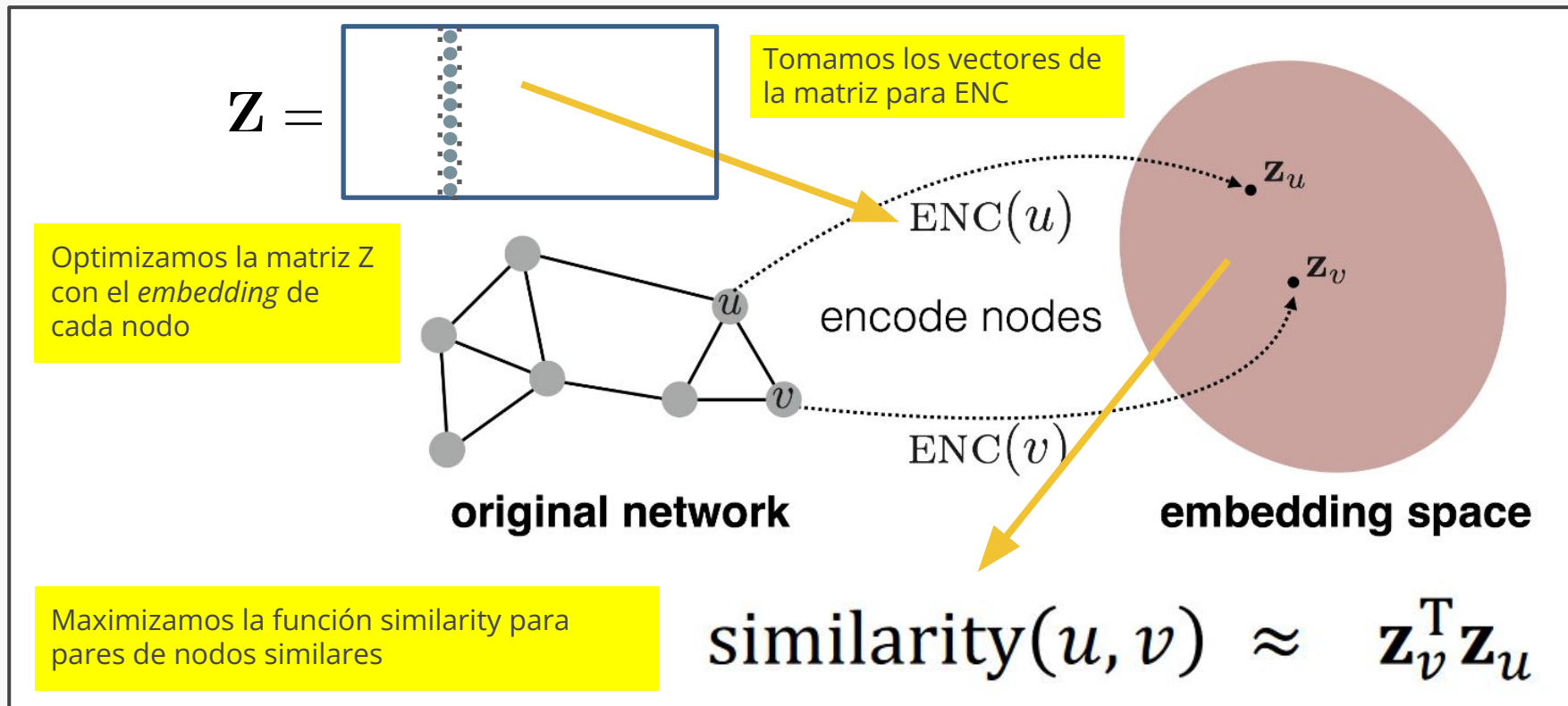
$$\mathbf{Z} \in \mathbb{R}^{d \times |\mathcal{V}|}$$

Matriz de *embeddings* (cada columna es el *embedding* de un nodo) **lo que aprendemos/optimizamos**

$$\mathbf{v} \in \mathbb{I}^{|\mathcal{V}|}$$

Vector *one-hot* que indica el índice del nodo v

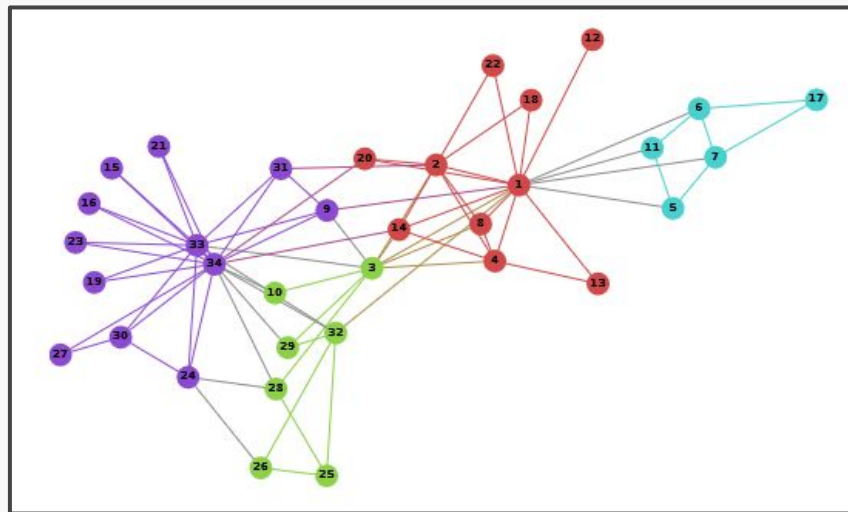
Resumen: Encoder + Decoder



¿Cómo definir similitud entre nodos?

¿Cuándo deben considerarse como similares dos nodos?

- ¿Si están conectados?
- ¿Si comparten vecinos?
- ¿Tienen “roles estructurales” similares?
- ¿...?



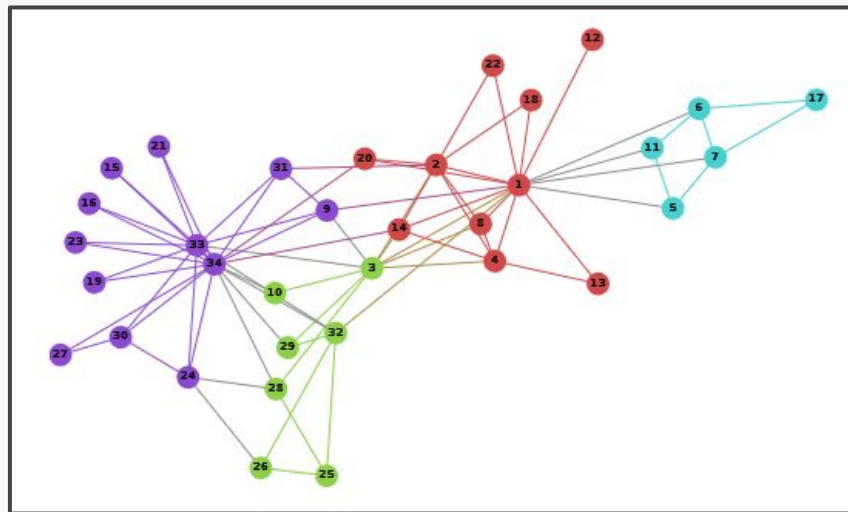
¿Cómo definir similitud entre nodos?

¿Cuándo deben considerarse como similares dos nodos?

- ¿Si están conectados?
- ¿Si comparten vecinos?
- ¿Tienen “roles estructurales” similares?
- ¿...?

En general, existen tres ideas principales:

1. Basada en adyacencia
2. Basada en múltiples *hops*
3. Basada en *random walks*



1 - Similitud basada en adyacencia

En la similitud basada en adyacencia, se utiliza el peso del arco entre los nodos como una medida de similitud: mayor peso significa mayor similitud

The diagram illustrates the loss function $\mathcal{L}$ for adjacency-based similarity. It is defined as the sum over all pairs of nodes $(u, v) \in V \times V$ of the squared L2 distance between the dot product of their embeddings and the weight of the edge between them. The components are labeled as follows:

- Función de pérdida**: Points to the symbol $\mathcal{L}$.
- Suma sobre todos los pares de nodos**: Points to the summation symbol $\sum_{(u,v) \in V \times V}$.
- Producto punto**: Points to the dot product term $\mathbf{z}_u^\top \mathbf{z}_v$.
- Matriz de adyacencia con pesos**: Points to the adjacency matrix element $\mathbf{A}_{u,v}$.

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \left\| \mathbf{z}_u^\top \mathbf{z}_v - \mathbf{A}_{u,v} \right\|^2$$

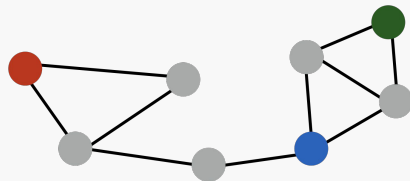
1 - Similitud basada en adyacencia

En la similitud basada en adyacencia, se utiliza el peso del arco entre los nodos como una medida de similitud: mayor peso significa mayor similitud

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \|\mathbf{z}_u^\top \mathbf{z}_v - \mathbf{A}_{u,v}\|^2$$

Algunas desventajas

- $O(|V|^2)$: considera todos los pares
- $O(|V|)$ parámetros, un vector por nodo
- Solo considera conexiones directas

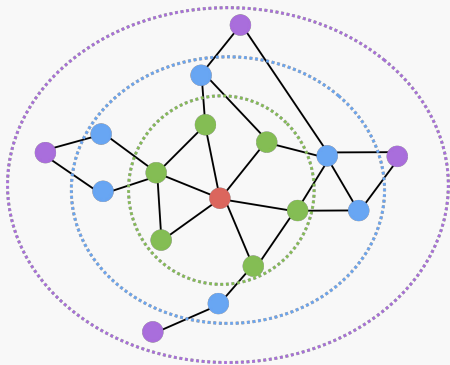


Nodo azul es más parecido al verde que el rojo, a pesar que ambos no están conectados a él

2 - Similitud basada en *hops*

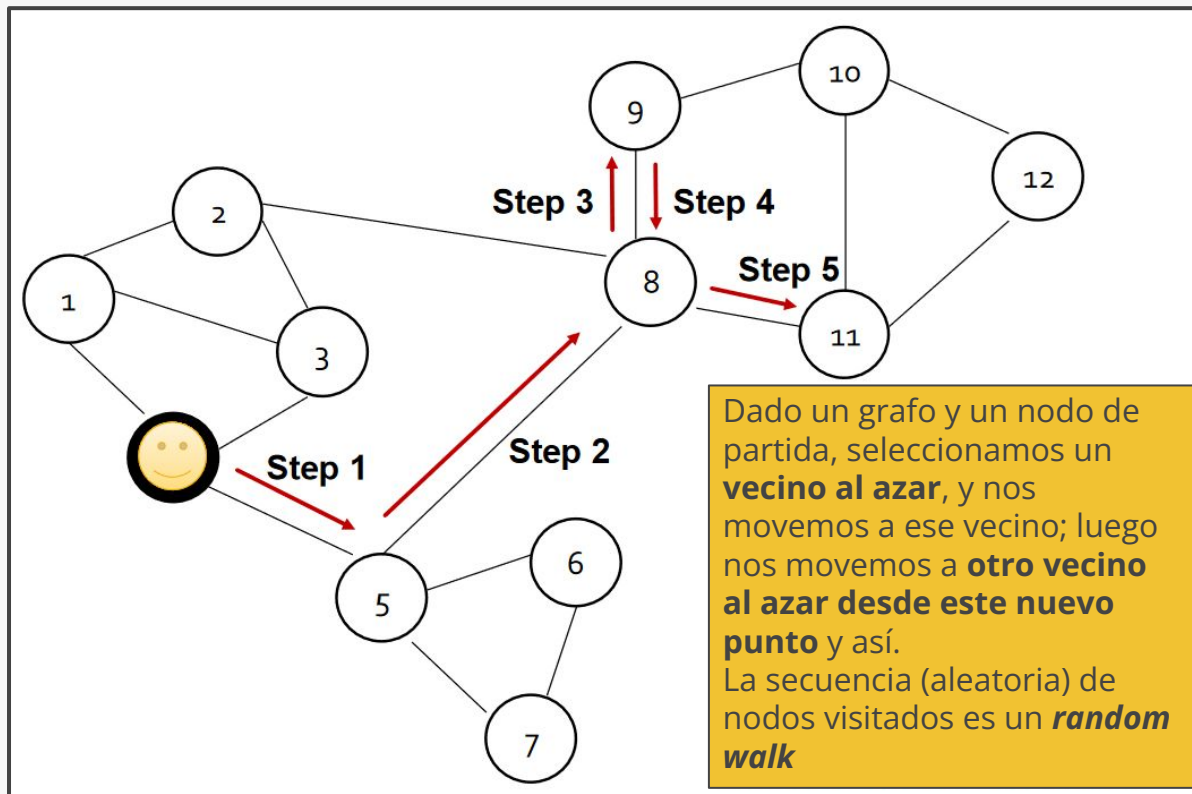
En la similitud basada en adyacencia, la similitud considera vecindarios de nodos a cierta distancia del nodo de interés

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \|\mathbf{z}_u^\top \mathbf{z}_v - \boxed{\mathbf{A}_{u,v}^k}\|^2$$



- **Rojo:** nodo objetivo
- **Verde:** vecindario de 1-hop (A)
- **Azul:** vecindario de 2-hop (A^2)
- **Morado:** vecindario de 3-hop (A^3)

3 - Similitud basada en... *random walks*?



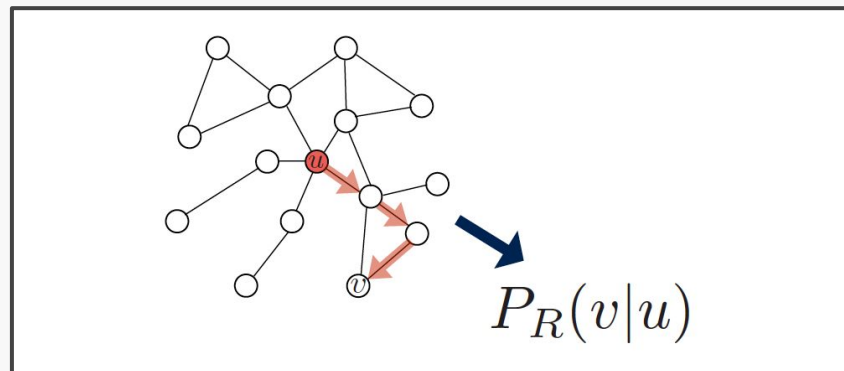
3 - Similitud basada en *random walks*

Similitud de *random walks* es la estrategia más exitosa actualmente

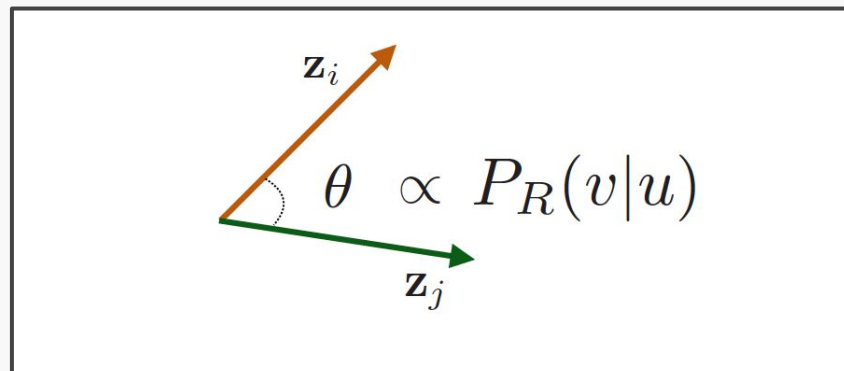
$$\mathbf{z}_u^\top \mathbf{z}_v \approx \text{Probabilidad que } u \text{ y } v \text{ co-ocurran en un } \textit{random walk} \text{ sobre el grafo}$$

3 - Similitud basada en *random walks*

1. Estimar la probabilidad de visitar al nodo v en un random walk partiendo de u , una estrategia de recorrido R .



2. Optimizar embedding para capturar probabilidades estimadas (producto punto = $\cos(\theta)$, "similaridad")



¿Por qué *random walks*?

1. **Expresividad:** definición flexible y estocástica de similitud que permite incorporar información de interacciones locales y de mayor orden.
Idea: si una *random walk* que parte del nodo u visita v con alta probabilidad, u y v son similares (información de mayor orden y múltiples hops)
2. **Eficiencia:** no necesita considerar todos los pares de nodos, solo aquellos que co-ocurren en los recorridos aleatorios.

¿Cómo funciona en la práctica?

1. Realizar recorridos cortos partiendo de cada nodos del grafo, usando alguna estrategia **R**
2. Para cada nodo u , recolectar $\mathbf{N}_R(u)$, el conjunto de nodos visitados en el recorrido desde u (posiblemente con nodos repetidos)
3. Optimizar embeddings usando SGD:

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v|\mathbf{z}_u))$$

¿Cómo funciona en la práctica?

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v|\mathbf{z}_u))$$

Intuición: optimizar *embeddings* para maximizar la probabilidad de co-ocurrencia en un recorrido, para lo que parametrizamos $P(v|z_u)$ usando *softmax* (queremos que el nodo v sea más similar al nodo u que a los demás)

$$P(v|\mathbf{z}_u) = \frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^\top \mathbf{z}_n)}$$

¿Cómo funciona en la práctica?

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log \left(\frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^\top \mathbf{z}_n)} \right)$$

Suma sobre todos los nodos u

Suma sobre todos los nodos v vistos en un recorrido comenzando en u

Probabilidad de co-ocurrencia en un recorrido

Generalizando *random walks*: node2vec

Objetivo: Crear un *embedding* de nodos donde nodos con vecindarios similares queden cerca en el espacio de features

Observación clave: Una noción flexible de vecindario en la red $N_R(u)$ permite generar *embeddings* ricos en información

Utilizaremos *random walks* R para generar vecindarios $N_R(u)$ del nodo u

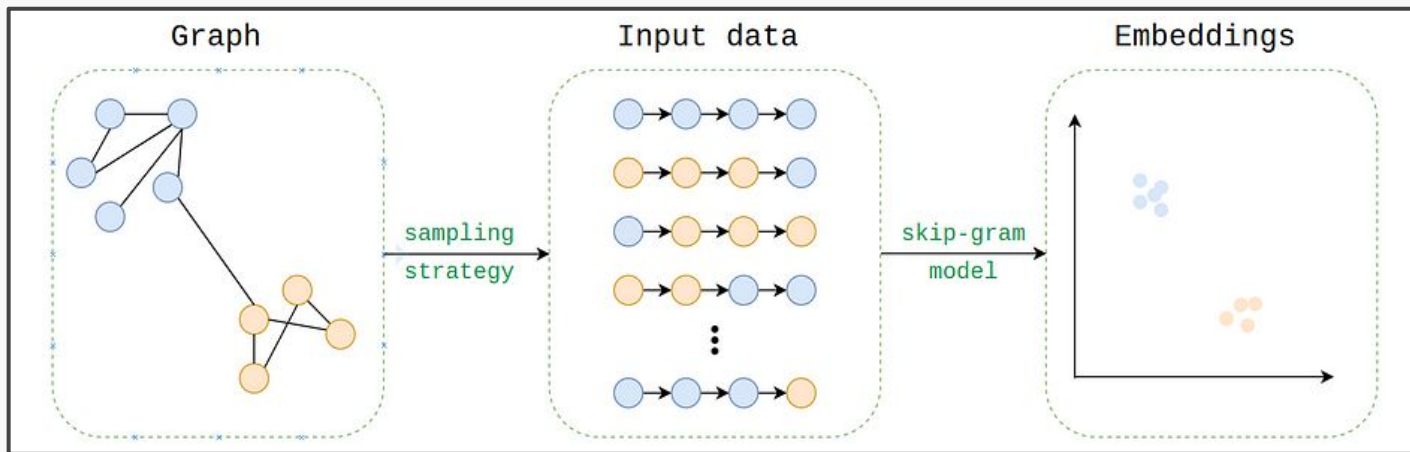
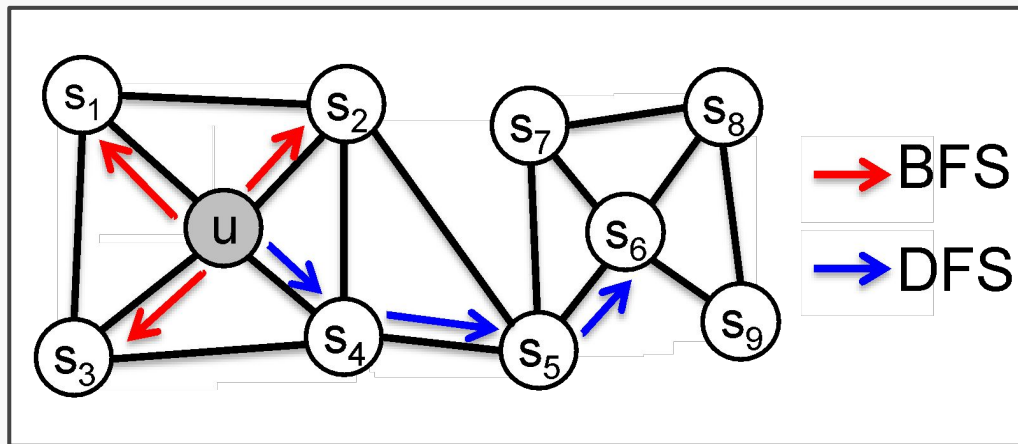


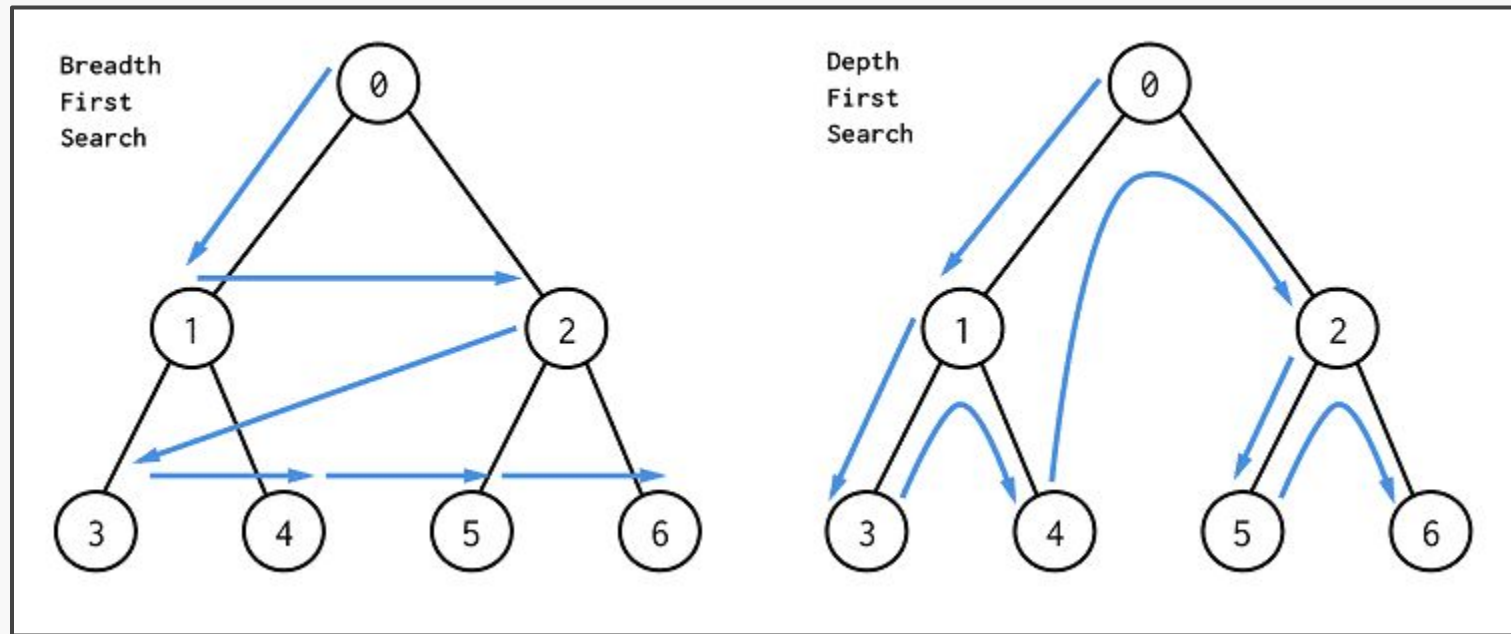
Figura: [node2vec: Embeddings for Graph Data | by Elior Cohen](#)

node2vec

Se basa en la idea de usar random walks sesgados, mezclando vistas **locales** (BFS) y **globales** (DFS) del grafo ([node2vec: Scalable Feature Learning for Networks](#))



(BFS vs DFS)



(BFS vs DFS)

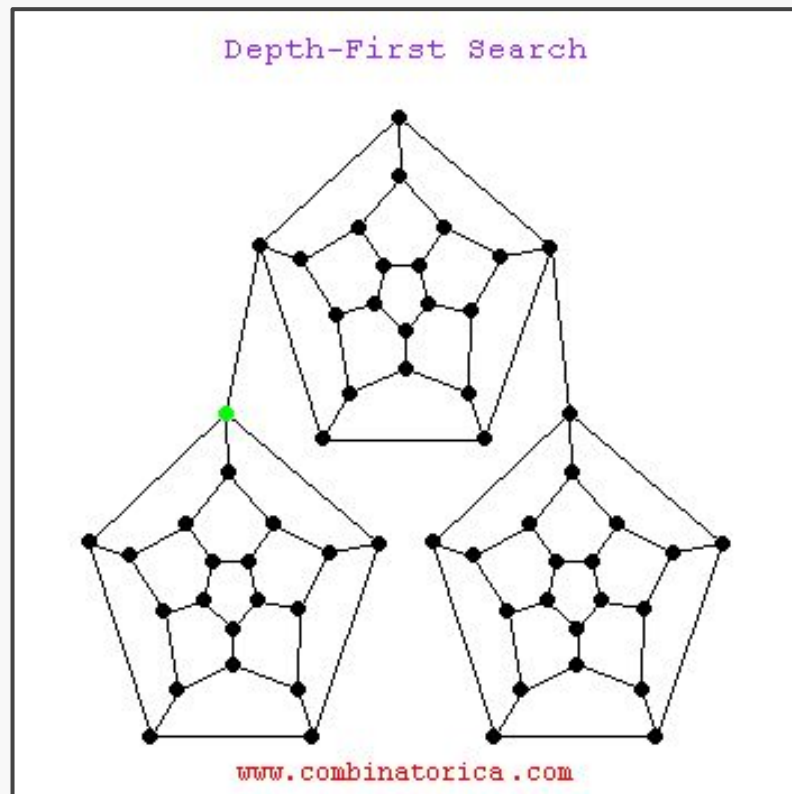
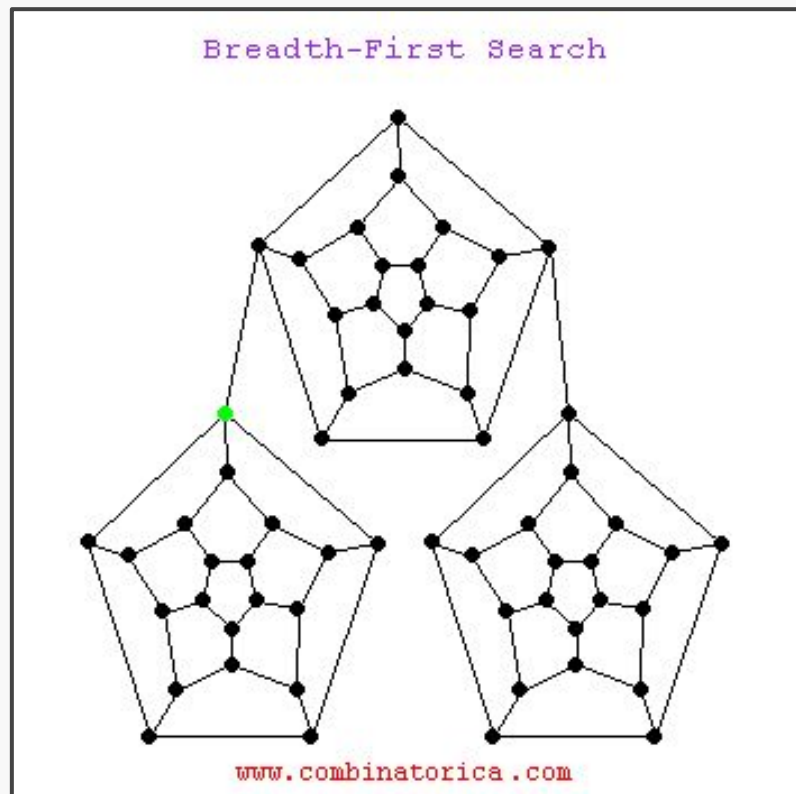
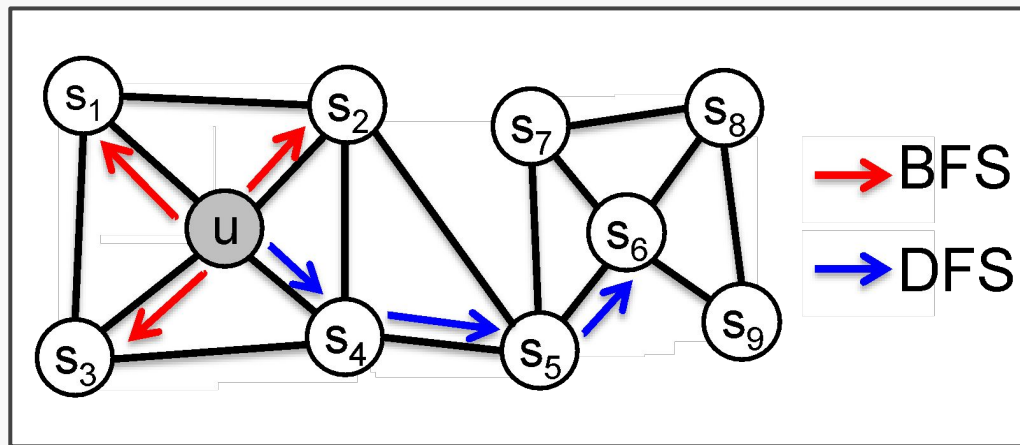


Figura: [Breadth First Search/Depth First Search Animations](#)

node2vec

Hay dos estrategias clásicas de definir el vecindario $N_R(u)$ de un nodo u :

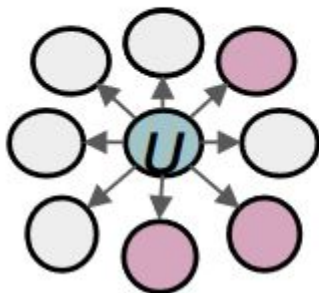


Camino de largo 3
($N_R(u)$ de tamaño 3)

$$N_{BFS}(u) = \{s_1, s_2, s_3\} \quad \text{Local}$$

$$N_{DFS}(u) = \{s_4, s_5, s_6\} \quad \text{Global}$$

BFS vs. DFS



BFS: Micro-vista de vecindario



DFS: Macro-vista de vecindario

Interpolando entre BFS y DFS

Biased random walk R de tamaño fijo que, dado un nodo u , genere un vecindario $N_R(u)$, donde el *bias* es introducido por los parámetros:

Parametro *return* (p):

- Volver al nodo previo

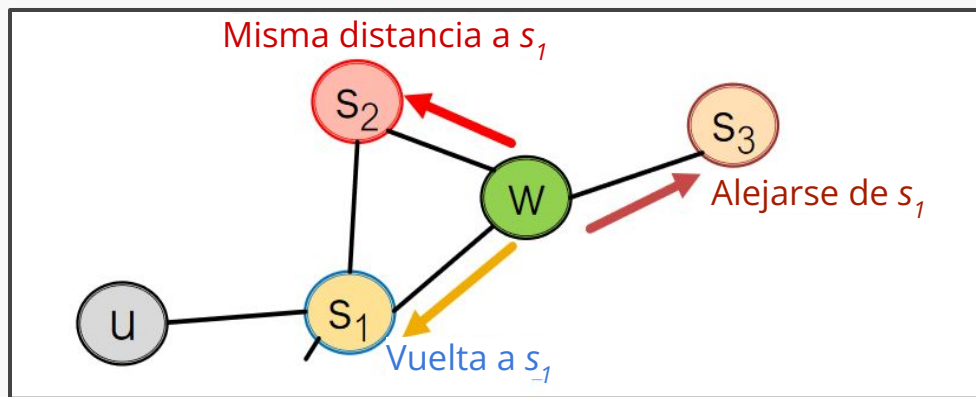
Parámetro *in-out* (q):

- Moverse hacia afuera (DFS) o hacia dentro (BFS)
- Es como un “ratio” o proporción entre BFS y DFS en cada paso

Biased random walks

Veámoslo con un ejemplo:

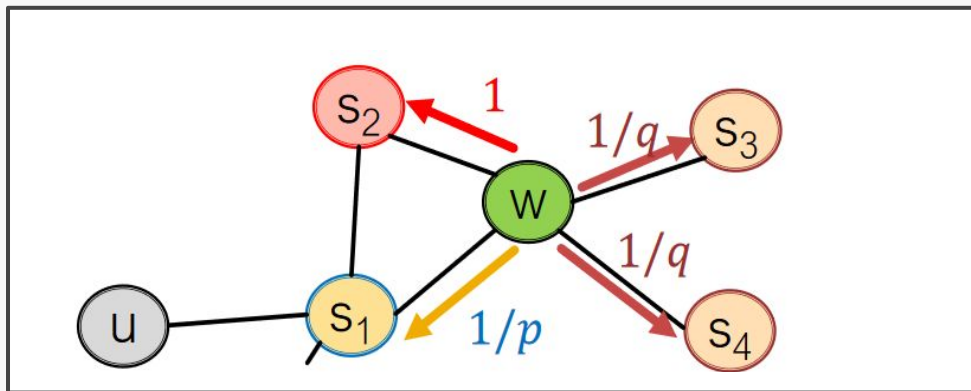
- Acabamos de hacer una *random walk* por el arco (s_1, w) y estamos en w
- Los vecinos de w , es decir, $N_R(w)$, solo pueden ser:



Biased random walks

Veámoslo con un ejemplo:

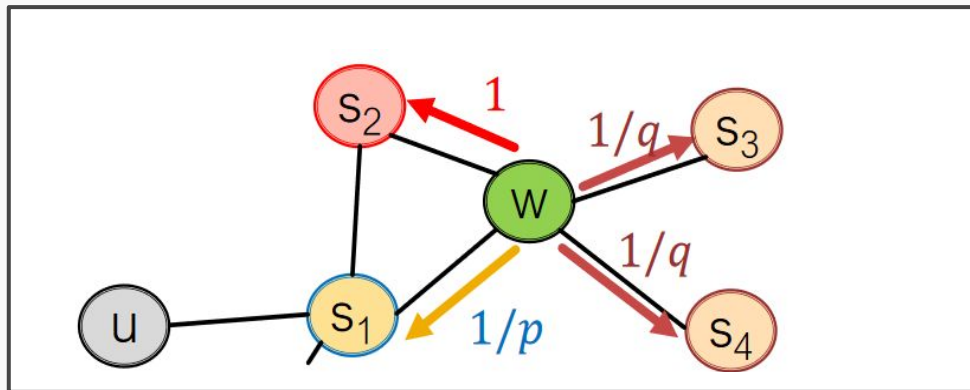
- Acabamos de hacer una *random walk* por el arco (s_1, w) y estamos en w
- Recordemos los parámetros:
 - p ... es el parámetro de retorno
 - q ... es el parámetro de alejamiento



Biased random walks

Veámoslo con un ejemplo:

- Acabamos de hacer una *random walk* por el arco (s_1, w) y estamos en w



- BFS-like:** valor bajo de p
- DFS-like:** valor bajo de q

$w \rightarrow$

Nodo t	Prob.	Dist. (s_1, t)
s_1	$1/p$	0
s_2	1	1
s_3	$1/q$	2
s_4	$1/q$	2

Probabilidades de transición no normalizadas, basadas en la distancia a s_1

Algoritmo node2vec

1. Calcular **probabilidades** de *random walks*
2. Simular *r random walks* de largo l empezando en cada nodo u
3. Optimizar el objetivo de node2vec usando **SGD**

¿Qué logramos?

Complejidad lineal. Además, cada paso es paralelizable. Mucho mejor que las otras opciones que consideramos inicialmente

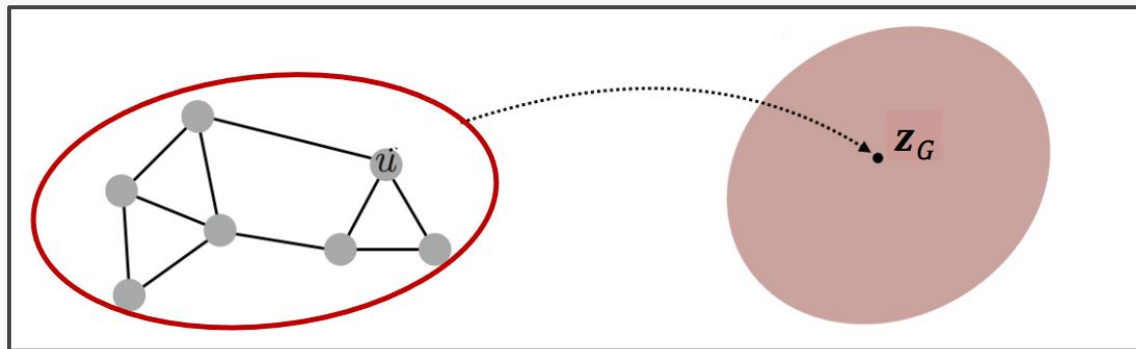
Agenda

Embeddings de nodos

✨ *Embeddings* de grafos

Embedding para todo un grafo

Objetivo: Queremos representar con un *embedding* a algún subgrafo o a todo un grafo G . Queremos poder crear el *embedding* z_G



¿Ejemplos de tareas?

- Clasificar moléculas (clasificación de grafos)
- Identificar grafos anómalos (lejos de la distribución)

Idea 1: agregar *embeddings* de nodos

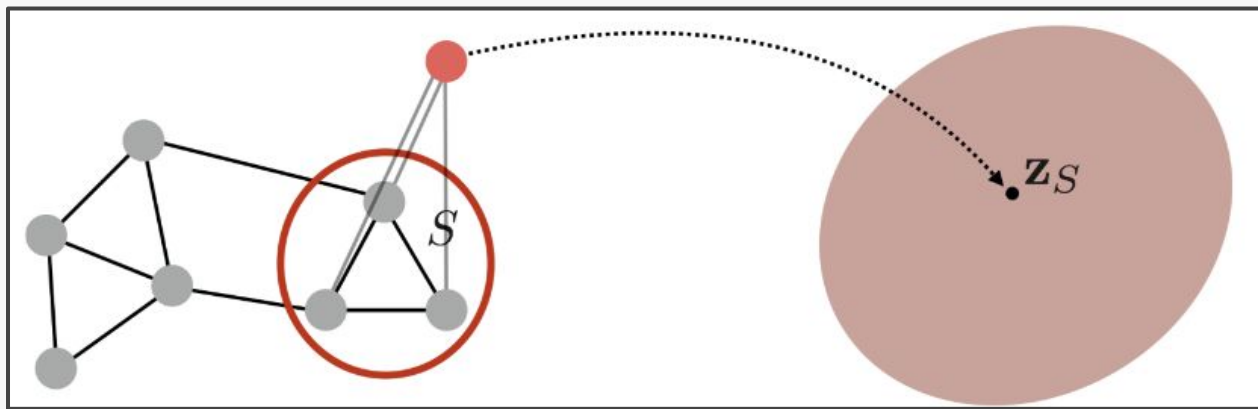
1. Aplicar una técnica estándar de *node embedding* sobre el (sub)grafo G
2. Luego sumar (o promediar) los *embeddings* en el (sub)grafo G

$$\mathbf{z}_G = \sum_{v \in G} \mathbf{z}_v$$

Idea utilizada para clasificación de moléculas a partir de la estructura del grafo que les representa ([\[1509.09292\] Convolutional Networks on Graphs for Learning Molecular Fingerprints](#))

Idea 2: agregar un “nodo virtual”

La idea es representar el (sub)grafo como si fuera un nodo y utilizar una técnica estándar de *embeddings* en grafos



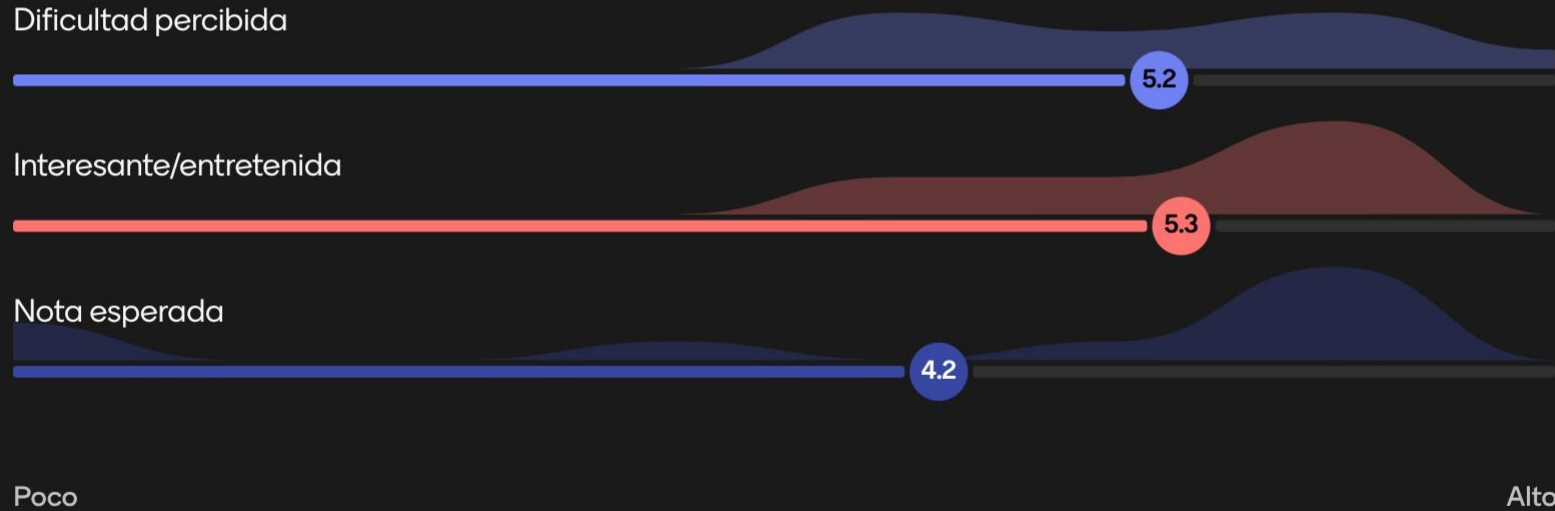
Propuesto por [\[1511.05493\] Gated Graph Sequence Neural Networks](#) como una técnica general para generar representaciones de subgrafos (*embeddings*)

Menti*#Hablemos sobre la EP1*

¿Cuánto llevan de la EP1?



¿Qué les pareció la EP1?



¿Comentarios de la EP1?

proporcionar una pauta cuando esté corregida la evaluación

Mas que difícil, extensa.

Extensa

Era interesante neo4j para visualizar, pero me queda la duda de si alguna vez lo usaré

La parte 2 de la tarea esta complicada, la parte de Neo4j y en la clase no vimos todo. Orientación

Volvamos al curso



¿Cuáles conceptos les gustaría repasar?

bases de datos

Casos de uso , ejemplos

Ejemplos

gRAFICO 3D :0 si es que existe

La parte de Neo4j la vimos muy rapido y la parte 2 esta complicado por ello!. Xd paso rapido la pregunta anterior

Respecto a la pregunta anterior, neo4j era interesante para visualizar pero no se que tanto se usa, y si la usaré algún día

Uso de neo4j

Lo del grafico 3D estaría bueno apoyo

¿Cuáles conceptos les gustaría repasar?

bases de datos

Casos de uso , ejemplos

Ejemplos

gRAFICO 3D :0 si es que existe

La parte de Neo4j la vimos muy rapido y la parte 2 esta complicado por ello!. Xd paso rapido la pregunta anterior

Respecto a la pregunta anterior, neo4j era interesante para visualizar pero no se que tanto se usa, y si la usaré algún día

Uso de neo4j

Lo del grafico 3D estaría bueno apoyo

Clase 5.1

Machine learning para grafos

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl